Rapport de l'Atelier 4

Les curseurs en MySQL

Environnement : MySQL 8.4 (Aiven Cloud) - Client en ligne de commande (cmd)

Novembre 2026

1. Objectif de l'atelier

Cet atelier a pour but de comprendre et d'utiliser les **curseurs** en MySQL. Un curseur permet de parcourir les résultats d'une requête **ligne par ligne** afin d'effectuer un traitement personnalisé sur chaque enregistrement, à l'intérieur d'une procédure stockée.

Le cycle d'utilisation d'un curseur comporte quatre étapes : **DECLARE** (déclarer), **OPEN** (ouvrir), **FETCH** (lire dans une boucle) et **CLOSE** (fermer). Un gestionnaire `CONTINUE HANDLER FOR NOT FOUND` permet de détecter la fin du jeu de résultats.

Le travail comprend un cas pratique (calcul de facturation d'eau) et un exercice (attribution de mentions aux étudiants).

2. Connexion au serveur MySQL

Depuis l'invite de commandes Windows, on se place dans le dossier *bin* de MySQL puis on se connecte au service MySQL hébergé sur Aiven avec la commande `mysql -h ... -P ... -u avnadmin -p --ssl-mode=REQUIRED defaultdb`. La commande `SELECT DATABASE();` confirme que la base active est **defaultdb**.

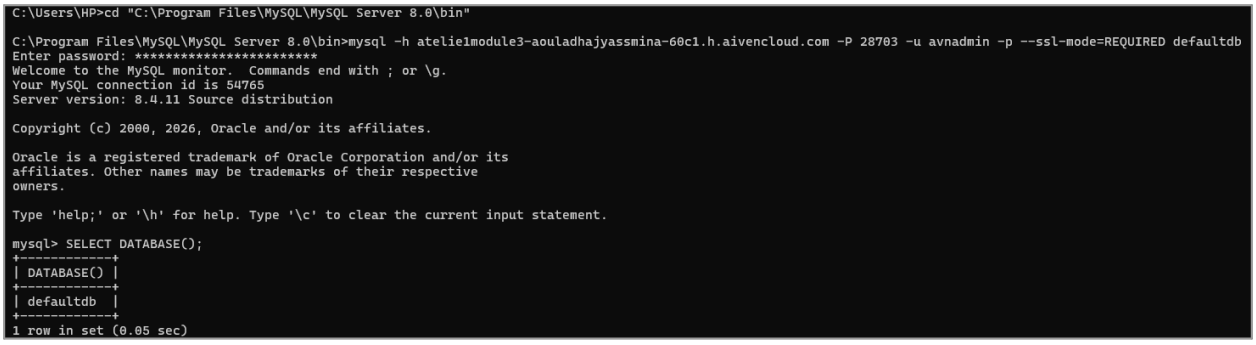


Figure 1 - Connexion au serveur et vérification de la base de données

3. Cas pratique : création de la table facture

On crée la table **facture** (id_facture, nom_client, qte_eau, montant) puis on y insère trois clients de test. La colonne *montant* est vide (NULL) : elle sera remplie par le curseur.

```
mysql> CREATE TABLE IF NOT EXISTS facture (
    ->   id_facture INT AUTO_INCREMENT PRIMARY KEY,
    ->   nom_client VARCHAR(100),
    ->   qte_eau INT,
    ->   montant DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql>
mysql> INSERT INTO facture (nom_client, qte_eau) VALUES
    -> ('Client 1', 9),
    -> ('Client 2', 15),
    -> ('Client 3', 25);
Query OK, 3 rows affected (0.10 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM facture;
+------------+------------+---------+---------+
| id_facture | nom_client | qte_eau | montant |
+------------+------------+---------+---------+
|          1 | Client 1   |       9 |    NULL |
|          2 | Client 2   |      15 |    NULL |
|          3 | Client 3   |      25 |    NULL |
+------------+------------+---------+---------+
3 rows in set (0.17 sec)
```

Figure 2 - Création de la table facture et insertion des données de test

4. Création de la procédure calculer_factures

La procédure déclare les variables, le curseur *curseur_facture* et le gestionnaire NOT FOUND. Dans la boucle, chaque ligne est lue avec FETCH, le montant est calculé (**qte_eau x 1,25**) puis enregistré avec UPDATE. La boucle se termine avec LEAVE lorsque la fin des données est atteinte, puis le curseur est fermé.

```
mysql> DELIMITER /
mysql> CREATE PROCEDURE calculer_factures()

    -> BEGIN
    ->   DECLARE v_id INT;
    ->   DECLARE v_qte INT;
    ->   DECLARE v_montant DECIMAL(10,2);
    ->   DECLARE fin BOOLEAN DEFAULT FALSE;
    ->
    ->   DECLARE curseur_facture CURSOR FOR
    ->     SELECT id_facture, qte_eau FROM facture;
    ->
    ->   DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = TRUE;
    ->
    ->   OPEN curseur_facture;
    ->
    ->   boucle_factures: LOOP
    ->     FETCH curseur_facture INTO v_id, v_qte;
    ->     IF fin THEN
    ->       LEAVE boucle_factures;
    ->     END IF;
    ->
    ->     SET v_montant = v_qte * 1.25;
    ->     UPDATE facture SET montant = v_montant WHERE id_facture = v_id;
    ->   END LOOP;
    ->
    ->   CLOSE curseur_facture;
    -> END /
Query OK, 0 rows affected (0.19 sec)
```

Figure 3 - Procédure stockée avec curseur pour le calcul des factures

5. Exécution de la procédure calculer_factures

On exécute la procédure avec `CALL calculer_factures();` puis on affiche la table. Les montants calculés sont : 9 x 1,25 = **11,25** ; 15 x 1,25 = **18,75** ; 25 x 1,25 = **31,25**.

```
mysql> DELIMITER ;
mysql> CALL calculer_factures();
Query OK, 0 rows affected (0.11 sec)

mysql> SELECT * FROM facture;
+------------+------------+---------+---------+
| id_facture | nom_client | qte_eau | montant |
+------------+------------+---------+---------+
|          1 | Client 1   |       9 |   11.25 |
|          2 | Client 2   |      15 |   18.75 |
|          3 | Client 3   |      25 |   31.25 |
```

Figure 4 - Résultat après exécution : la colonne montant est renseignée

6. Exercice : création de la table notes

On crée la table **notes** (id_note, nom_etudiant, note, mention) et on insère quatre étudiants avec leurs notes.

```
mysql> CREATE TABLE notes (
    ->   id_note INT AUTO_INCREMENT PRIMARY KEY,
    ->   nom_etudiant VARCHAR(100),
    ->   note DECIMAL(4,2),
    ->   mention VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.18 sec)
mysql>
mysql> INSERT INTO notes (nom_etudiant, note) VALUES
    -> ('Ahmed', 17), ('Sara', 14.5), ('Youssef', 11), ('Salma', 7);
Query OK, 4 rows affected (0.13 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

Figure 5 - Création de la table notes et insertion des données

7. Création de la procédure attribuer_mentions

Le curseur parcourt chaque étudiant. Selon la note, une mention est attribuée avec une structure IF / ELSEIF : **Excellent** (note >= 16), **Bien** (>= 14), **Passable** (>= 10) et **Insuffisant** (< 10). La colonne *mention* est mise à jour pour chaque ligne.

```
mysql> DELIMITER /
mysql> CREATE PROCEDURE attribuer_mentions()
    -> BEGIN
    ->   DECLARE v_id INT;
    ->   DECLARE v_note DECIMAL(4,2);
    ->   DECLARE fin BOOLEAN DEFAULT FALSE;
    ->
    ->   DECLARE cur CURSOR FOR SELECT id_note, note FROM notes;
    ->   DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = TRUE;
    ->
    ->   OPEN cur;
    ->   boucle: LOOP
    ->     FETCH cur INTO v_id, v_note;
    ->     IF fin THEN LEAVE boucle; END IF;
    ->
    ->     IF v_note >= 16 THEN
    ->       UPDATE notes SET mention = 'Excellent' WHERE id_note = v_id;
    ->     ELSEIF v_note >= 14 THEN
    ->       UPDATE notes SET mention = 'Bien' WHERE id_note = v_id;
    ->     ELSEIF v_note >= 10 THEN
    ->       UPDATE notes SET mention = 'Passable' WHERE id_note = v_id;
    ->     ELSE
    ->       UPDATE notes SET mention = 'Insuffisant' WHERE id_note = v_id;
    ->     END IF;
    ->   END LOOP;
    ->   CLOSE cur;
    -> END /
Query OK, 0 rows affected (0.39 sec)
```

Figure 6 - Procédure stockée avec curseur pour l'attribution des mentions

8. Exécution et vérification du résultat final

Après `CALL attribuer_mentions();`, la requête `SELECT * FROM notes;` montre que chaque étudiant a reçu la mention correspondant à sa note.

```
mysql> DELIMITER ;
mysql> CALL attribuer_mentions();
Query OK, 0 rows affected (0.10 sec)

mysql> SELECT * FROM notes;
+---------+-------------+-------+-------------+
| id_note | nom_etudiant | note  | mention     |
+---------+-------------+-------+-------------+
|       1 | Ahmed       | 17.00 | Excellent   |
|       2 | Sara        | 14.50 | Bien        |
|       3 | Youssef     | 11.00 | Passable    |
|       4 | Salma       |  7.00 | Insuffisant |
+---------+-------------+-------+-------------+
4 rows in set (0.44 sec)
```

Figure 7 - Résultat final de la table notes avec les mentions

9. Conclusion

Cet atelier a permis de maîtriser le cycle complet d'un curseur MySQL (DECLARE, OPEN, FETCH, CLOSE) ainsi que la gestion de la fin de lecture avec un gestionnaire NOT FOUND. Les deux procédures réalisées (calcul des factures et attribution des mentions) ont produit les résultats attendus :

Étudiant	Note	Mention
Ahmed	17	Excellent
Sara	14,5	Bien
Youssef	11	Passable
Salma	7	Insuffisant